KAS SIGNER

kassigner.org

Donate: kassigner.kas

KasSigner + KasSee

Covenants++ & Stealth Payments

Programmable spending conditions and unlinkable payments for Kaspa. cov++ runs in KasSee for building and watching, and on KasSigner for signing. Your keys never leave the device.

Release v1.0.7

Date August 2026

Licence GPL-3.0

Repository github.com/InKasWeRust/KasSigner

Contact kassigner@proton.me

Covenants and stealth payments are advanced features. A covenant locks funds under on-chain script rules that only the covenant itself can release; a mistake in setup can lock funds until a timeout, or permanently. Stealth scanning depends on on-chain announcements that may be pruned. Test every covenant type with small amounts before committing real value, and always verify the destination and the P2SH address on your KasSigner screen before signing.

Don't trust, verify. Sign offline. Broadcast sovereign.

Contents

1. What Covenants Are (cov++)

A covenant is a set of on-chain rules that control how funds can be spent, enforced by the Kaspa script engine itself rather than by a single signature. Instead of "whoever holds the key can spend," a covenant says things like "this can only be spent to an approved address," "only after a certain time," "only up to a set amount per period," or "only when an oracle attests to a price."

cov++ is KasSigner's covenant suite for Kaspa after the Toccata network upgrade (script introspection, OP_CAT, the ZK precompile), live on mainnet. Covenant funds live at a Pay-to-Script-Hash (P2SH) address whose script encodes the rules. Anyone can pay into that address like any normal Kaspa address. Spending from it requires a transaction that satisfies the script, which KasSee builds and KasSigner signs.

Two roles, two tools:

- **KasSee** (browser, watch-only) builds the covenant redeem script, computes the P2SH address, constructs deposit and spend transactions, and watches the covenant on-chain. It never holds keys.
- **KasSigner** (air-gapped device) reviews the transaction on its own trusted screen and signs it. For no-signature covenant branches, no signing is needed at all; the script rules alone authorize the spend.

cov++ transactions use Kaspa transaction version 1 for covenant-binding outputs and version 0 for plain sends. KasSee handles this; you never set it by hand.

2. How a Covenant Works: Build, Verify, Sign

Every covenant follows the same lifecycle. Understanding it once applies to all types.

1. **Build.** In KasSee, choose a covenant type and fill in its parameters (addresses, amounts, timeouts, oracle keys, and so on). KasSee assembles the redeem script and derives the P2SH address: address = bech32(blake2b(redeem_script)).
2. **Fund.** Send KAS to that P2SH address from any wallet. Deposits are ordinary sends; no special support is needed on the sending side.
3. **Spend.** When the rules allow, KasSee builds a spend transaction whose inputs reference the covenant UTXO and whose sig_script provides the script branch plus any required data (signatures, preimages, oracle attestations).
4. **Verify and sign.** KasSigner scans the transaction, checks that the redeem script hashes to the P2SH address being spent, and shows every input and output, the amount, the fee, the payload hash and the covenant id on its screen. You confirm, and it signs the branch that needs a signature. No-signature branches broadcast directly from KasSee.

> **WARNING:** The redeem script and its P2SH address are the whole agreement. Verify the P2SH address on the KasSigner review screen before you deposit meaningful funds, and exchange the initial public keys out-of-band. A covenant_id label alone does not prevent a forged QR; the trust root is the address shown on the device plus the out-of-band key exchange.

3. The Covenant Menu

Open KasSee, load your kpub, and tap Covenants on the dashboard. The covenant cards are grouped into four categories. Each card below shows exactly what the app shows.

Savings & Time Locks

- **Dead Man's Switch**: heir inherits after inactivity. Heartbeat resets timer.
- **Piggy Bank**: save toward a goal or a deadline. Break it open to withdraw.
- **Time-Locked Savings**: lock funds until a date. No one can touch them early, not even you.

Spending Controls

- **Allowance**: a beneficiary withdraws up to a cap, with a cooldown between withdrawals.
- **Merkle Whitelist**: spend to an approved set via proof.
- **Spending Limit**: a cap on each withdrawal, with a cooldown between them. Applies to your whole balance.

Multi-Party & Conditional

- **Direct Channel**: direct payment channel with arbiter dispute resolution.
- **Oracle**: release on oracle attestation.
- **PayJoin**: anonymous payment covenant.

Advanced & PoC

- **Commit-Reveal**: MEV-resistant inscriptions.
- **Private Swap**: private atomic swap. No preimage, no on-chain link.
- **ZK Price Oracle**: live KAS/USD. Pyth + Wormhole, proven on-chain.

Each category can be collapsed or expanded with the arrow on its header. Tap a card to open that covenant's build screen.

4. Savings & Time Locks

These protect your own funds over time.

Dead Man's Switch

A designated heir inherits the funds after a period of inactivity. You send a periodic heartbeat that resets the timer; as long as you are active, only you control the funds. If the heartbeat stops for the set duration, the heir's claim path opens. Useful for inheritance and continuity planning without handing anyone your keys.

Piggy Bank

Deposit any amount, any time, from any wallet, saving toward a goal or a deadline. To take the funds out you "break it open"; withdrawal is to your own address. Deposits are ordinary sends to the P2SH address.

Time-Locked Savings

Funds are locked until a chosen date and cannot move before it, not by anyone, including you. Enforced on-chain by CheckLockTimeVerify. After the unlock date you sign and spend normally.

> **WARNING:** Dead Man's Switch and Time-Locked Savings make funds temporarily unspendable by design. Note the exact heartbeat interval or unlock date when you create one. Save the covenant (locally, to KasSigner, or recover it from the chain later) so you always have its parameters; see section 8.

5. Spending Controls

These cap how funds leave a covenant, enforced by the script rather than by trust.

Allowance

A beneficiary may withdraw up to a cap, with a cooldown between withdrawals. You set the cap, the cooldown, and the beneficiary when you create it. The cooldown is enforced on-chain by CheckSequenceVerify, so the beneficiary cannot drain everything at once.

> **WARNING:** When entering the beneficiary for an Allowance, provide an individual address, not an account-level extended public key (kpub). A kpub exposes the whole account. KasSee warns you if a scanned value looks like a kpub.

Merkle Whitelist

Spending is allowed only to an approved set of addresses, proven at spend time by a Merkle branch. You enter the whitelist (one address per line) when you create the covenant. This is a core building block for autonomous-agent wallets: an agent can move funds, but only to addresses you approved in advance.

Spending Limit

A cap on each withdrawal, with a cooldown between them, applied to your whole balance. Combine the per-spend cap with the cooldown for a policy wallet that limits how fast funds can leave regardless of who triggers the spend.

6. Multi-Party & Conditional

These involve more than one party or an external condition.

Direct Channel

A direct payment channel between two parties with arbiter dispute resolution. The two parties transact directly off the main flow, and a designated arbiter can resolve a dispute if they disagree, without custody of the funds.

Oracle

Funds release on an oracle attestation, a signed statement from a trusted data source that a condition is met. The covenant checks the attestation's signature before allowing the spend, so release is automatic once the oracle attests.

PayJoin

An anonymous payment covenant. It structures a payment so that on-chain analysis cannot cleanly attribute inputs to a single payer, improving payment privacy.

> **WARNING:** Oracle covenants depend on an external data source. Verify the oracle's public key and the freshness of its attestation before relying on an automatic release. Treat conditional covenants as advanced and test with small amounts first.

7. Advanced & PoC

Advanced and proof-of-concept covenants. Treat these as experimental.

Commit-Reveal

A two-phase commit-reveal scheme for MEV-resistant inscriptions. You first commit to data with a hash, then reveal it in a later transaction, so the content cannot be front-run or reordered for advantage between commit and reveal. The device side lives under Tools, Single Signature: Commit Secret and Decrypt Secret.

Private Swap

A private atomic swap of KAS for KAS between two parties, with no preimage and no on-chain link between the two legs. One side creates the swap, the other joins it; the swap completes without publishing a shared secret, so observers cannot connect the two trades. A refund path returns funds if the counterparty never completes. The swap is KAS/KAS only.

ZK Price Oracle

Brings the real KAS/USD price on-chain as a zero-knowledge proof, sourced from Pyth and delivered via Wormhole. The proof is verified on-chain, so the price is trustless: you are not trusting a third-party price server, you are checking a proof against on-chain data. It provides the verified price itself; it does not move or release funds. Other covenants or applications can then use that proven price as a condition.

> **WARNING:** Verify the attestation freshness before relying on a price, and test with small amounts before trusting real value.

8. Creating a Covenant (Step by Step)

The flow is the same for every type; only the parameters differ.

- In KasSee, tap Covenants, then the card for the type you want.
- Fill in the parameters. Common fields: your own address (owner), a beneficiary or whitelist, a maximum amount, a timeout (date/time), and for oracle types the oracle public key.
- Tap Create. KasSee builds the redeem script and shows the covenant's P2SH address and its covenant_id.
- Verify the P2SH address. This address is where funds will live. Compare it against what KasSigner shows on its review screen when you first interact with the covenant.
- Save the covenant. KasSee keeps it in your Active Covenants list so it can rebuild spends later.

Saving and restoring a covenant

A covenant is defined by its parameters (type, keys, amounts, timeouts); KasSee needs them to rebuild the exact redeem script and spend. cov++ gives you three ways to keep and recover a covenant so you are never dependent on browser storage alone:

- **Save locally**: export the covenant to a file (and QR) from KasSee.
- **Save to KasSigner**: hand the covenant to the device via QR; it is stored on the device's SD card as a Covenant Restore file and loaded back through Tools, Import / Export, Covenant Restore.
- **Recover from mainnet**: rebuild a covenant from its on-chain data via the REST API, so even with no local copy you can restore it from the chain.

Because a covenant can be exported locally, stored on KasSigner, and recovered from the chain, clearing browser storage does not lose it. Still, keeping your own export is good practice for the fastest restore.

9. Depositing to a Covenant

Depositing is just sending KAS to the covenant's P2SH address. It works from KasSee, from KasSigner via a normal signed send, or from any other Kaspa wallet.

- Copy the covenant P2SH address from the Active Covenants list.
- Send KAS to it as you would to any address.
- The deposit appears as a UTXO at the covenant address once confirmed.

Kaspa's storage-mass rule (KIP-9) blocks tiny data-only UTXOs. Deposit meaningful amounts, not dust; KasSee warns when a balance is very low. Covenant spends carry a higher fee than plain sends because each input's script costs compute mass; KasSee computes it from the number of inputs and sets it automatically, so you do not enter it.

10. Spending From a Covenant

Spending must satisfy the covenant's rules. KasSee picks the correct script branch based on what you are trying to do and builds the transaction; KasSigner signs the branch that needs a signature.

- Open the covenant in Active Covenants and choose the action (withdraw, claim, refund, release).
- KasSee builds the spend transaction, embedding the script branch and any required data (signature, preimage, oracle attestation, Merkle proof).
- For branches that need a signature, KasSee shows the transaction as a QR. Scan it with KasSigner.
- On KasSigner, review every output, the amount, the fee and the lock time if the spend carries one, then CONFIRM. The device signs and shows the signed QR.
- In KasSee, scan the signed QR (or use the no-signature path) and broadcast.

No-signature branches. Some covenant branches are authorized purely by the script rules (for example, an anyone-can-claim timeout). These need no KasSigner signature at all; KasSee builds and broadcasts them directly. This is what makes autonomous-agent covenants possible: the agent holds no key and simply produces a transaction the rules already permit.

> **WARNING:** Always confirm the destination address on the KasSigner screen matches your intent. For covenant spends the destination may itself be another P2SH (a continuation of the covenant); the device shows a "To P2SH" tag so you can tell a covenant continuation from a payout.

11. Watching Active Covenants

KasSee keeps your covenants in an Active Covenants list and watches them on-chain so you see deposits, spends, and state changes.

- Each entry shows the covenant type, its P2SH address, current balance, and covenant_id.
- KasSee polls balances and subscribes to new blocks so a spend or deposit shows up promptly.
- The watcher uses the node's BlockAdded stream to detect when a covenant UTXO is spent, and reads the spending transaction to follow the covenant's lineage.

Covenant watching runs while KasSee is open and connected. If you close the page, the live watch stops; reopening KasSee re-reads current state from the node.

12. Covenant Verification & Trust Model

Covenants move trust from a person to a script, but you still have to verify the script is the one you agreed to. The trust model rests on three things:

- **The P2SH address on the KasSigner screen.** Before signing, the device checks that the redeem script the host supplied hashes to the address being spent, so a substituted script cannot vouch for itself. If the address matches what you verified at creation, the rules are what you expect.
- **Out-of-band key exchange.** For multi-party covenants, exchange public keys through a channel other than the covenant QR (in person, signed message) so a swapped key is caught.
- **covenant_id for lineage, not authenticity.** It tracks a covenant's history but is recomputable, so it does not by itself prove a QR is genuine. The device shows the covenant_id and the payload hash on its review screen so you can check them against what you expect and, after broadcast, against what landed on-chain.

> **WARNING:** Never deposit real funds to a covenant address you have not verified on the KasSigner screen. A malicious companion could show one address while the real script pays elsewhere. The device is the only trusted display.

13. What Stealth Addresses Are

A stealth address lets someone pay you without the payment being publicly linked to your wallet. You publish one reusable meta-address. Each sender derives a fresh, unique one-time address from it, so no two payments to you share an address on-chain and outside observers cannot tell they went to the same person.

KasSigner's stealth system uses dual-key ECDH (a DKSAP-style scheme). You have a scan key and a spend key. The sender combines your published keys with a random ephemeral value R to compute a one-time public key that only you can find and only you can spend.

- **Meta-address**: what you publish. 128 hex characters (two public keys).
- **R value**: the sender's random ephemeral point, announced on-chain so you can find the payment.
- **One-time address**: the unique address each payment actually lands on.

Stealth payments protect the recipient's privacy. The amount and the one-time address are still visible on-chain like any Kaspa transaction; what is hidden is the link between the payment and your published identity.

14. Your Stealth Meta-Address

Open KasSee, load your kpub, and tap Stealth Addresses, then My Stealth Meta-Address. KasSee shows your 128-hex meta-address and its QR.

- Share the meta-address (or QR) with anyone who should be able to pay you privately.
- It is reusable: one meta-address serves for all senders and all payments.
- It reveals nothing about your balance or your normal addresses.

> The meta-address is derived from your scan and spend public keys. It is safe to publish. Your spend private key never leaves KasSigner; KasSee only holds public material, so it can find incoming stealth payments but cannot spend them without the device.

15. Sending to a Stealth Address

To pay someone at their stealth meta-address, in KasSee tap Stealth Addresses then Send to Stealth Address.

- Paste or scan the recipient's 128-hex meta-address.
- Enter the amount.
- KasSee generates a random ephemeral R, derives the recipient's one-time address, and builds the transaction.
- The one-time address is shown so you can see where the funds are going.
- The R value is announced in the transaction payload (lane tag KST1) so the recipient can discover the payment.
- Sign on KasSigner as with any send, then broadcast from KasSee.

Each send produces a new one-time address, even to the same recipient, so repeated payments are unlinkable on-chain.

16. Scanning for Stealth Payments

Incoming stealth payments do not appear at your normal addresses. You find them by scanning the announced R values and checking which derived one-time addresses hold funds. In KasSee, tap Stealth Addresses then Scan for Stealth Payments.

- Tap **1. Fetch Announcements**. KasSee scans recent on-chain announcements for R values on the KST1 lane.
- It scans approximately the last 15 minutes on-chain by default, then keeps a live watch on new blocks while the page stays open.
- For each R found, KasSee sends the R values to KasSigner (via QR) for ECDH scanning, and checks which one-time public keys have UTXOs.
- Found payments appear in the list; tap Show QR for Device Scan to hand the R set to KasSigner.

Manual R entry. If you received an R value out-of-band, paste the 64-hex value into the Manual R Entry field and tap Add R Value. This is the fallback when an announcement was pruned or missed.

Stealth Indexer (optional). In Settings you can enable a Stealth Indexer pointing at a keeper service that pre-scans the lane and serves R values in one request, instead of scanning in-browser. This removes the per-scan load on public infrastructure.

The live scan only catches payments announced while KasSee is open and connected. The Fetch Announcements catch-up backfills the recent window. Payments older than the catch-up window that were missed live need a wider scan or the indexer.

17. Spending a Received Stealth Payment

Once KasSee has found a stealth payment, spending it works like any send; the difference is that KasSigner derives the one-time private key for that specific payment.

- KasSee hands the R set and the funded one-time addresses to KasSigner.
- KasSigner uses your spend key plus each R to derive the one-time private key via ECDH.
- Build a normal spend in KasSee to your chosen destination; it uses the stealth UTXO as input.
- Review on KasSigner and sign; broadcast from KasSee.

Because the one-time private key is derived on the device from your spend key and the sender's R, KasSee never sees it. The device is required to spend a stealth payment.

18. Stealth Scan Tuning & Limits

Stealth catch-up scans the chain in fixed windows against a public REST endpoint, which rate-limits per IP. KasSee is tuned to stay under that limit.

- The default scan covers about the last 15 minutes of chain history, which completes cleanly against public infrastructure without hitting rate limits.
- Requests are paced and retried with backoff so a brief throttle does not drop a window.
- For deeper history, run your own kaspa-rest-server and point the Stealth Indexer at it, or use a REST proxy on your own origin. A self-hosted indexer removes the per-IP limit entirely.

The live scan (BlockAdded subscription) has no window limit; it catches every new announcement in real time while KasSee is open. The window limit only affects how far back the one-time catch-up reaches.

> **WARNING:** Public REST endpoints are best-effort with no guarantees. For reliable deep scanning, and for privacy, run your own node and indexer. Scanning through a shared public endpoint means your queries traverse third-party infrastructure.

19. Security Model

Covenants and stealth payments inherit KasSigner's core model: keys live only on the air-gapped device, KasSee is watch-only, and the device screen is the only trusted display.

- **Keys never leave KasSigner.** KasSee holds public material only. It can build and watch, but cannot sign or spend.
- **Verify on device.** Covenant P2SH addresses and spend destinations must be confirmed on the KasSigner screen; the device checks the redeem script against the address before it signs.
- **Covenant rules are on-chain.** Once funds are at a verified covenant address, the network enforces the rules; no one, including you, can bypass them except as the script allows.
- **Stealth spend keys stay on device.** KasSee can discover incoming stealth payments from public data but needs KasSigner to derive the one-time keys and spend.
- **Out-of-band exchange.** For multi-party covenants and stealth meta-addresses, share keys through a channel separate from the covenant/payment QR to defeat substitution.

> **WARNING:** cov++ and stealth are advanced features and have not been professionally audited. Test each covenant type and the stealth flow with small amounts before trusting real value, and keep independent records of every covenant's parameters.

20. Troubleshooting

Covenant spend rejected by the network

The transaction did not satisfy the script. Check that the branch conditions are met: a time lock has passed, an amount is within the limit, the destination is on the whitelist, or the oracle attestation is fresh and correctly signed. KasSee rebuilds the spend from the saved covenant parameters; if the covenant is missing, restore it from your local export, from KasSigner, or from the chain via the REST API (section 8).

Covenant address does not match on KasSigner

Do not deposit. A mismatch means the script KasSee built is not the one the device derived. Recreate the covenant and compare again. The device address is authoritative.

Stealth scan finds no payments

The catch-up window is about the last 15 minutes; a payment older than that, missed by the live scan, needs a wider scan or an indexer. Confirm the sender actually used your current meta-address. Try Manual R Entry if you have the R value out-of-band.

Stealth scan shows rate-limit errors

The public REST endpoint throttled the scan. The default 15-minute window is tuned to avoid this; if you widened it, lower it again, or run your own indexer. The live scan is unaffected.

Cannot spend a found stealth payment

Spending requires KasSigner to derive the one-time key from your spend key and the sender's R. Make sure the correct seed is active on the device and that KasSee handed over the right R set.

Covenant funds appear stuck

Time-locked and goal covenants are unspendable until their condition is met; this is by design, not a fault. Check the exact unlock condition you set. If a covenant has a recovery path (CheckLockTimeVerify), it becomes available after the recovery timeout.